

# Computable Countermarkets and the Limits of Universal Trading

## A Computable Countermarket for Every Trader

Karl Svozil  
ITP, TU Wien, Vienna, Austria

ECONVN 2026/2027, HUB/AJEB,  
Ho Chi Minh City, Vietnam  
12-14 October 2026

<https://svozil.github.io/publications/2026-ECONVN-pres.pdf>  
based on  
<https://doi.org/10.48550/arXiv.2604.13334>

# Finite profits do not establish universal profitability

## A trader can make money over a finite period

Accurate information, luck, or a continuing trend can produce profits. Exuberance, innovation, and improvements in production can support a rise.

## The same kind of bet can also lose

Information may be false. Exuberance may turn into panic. Expected gains from innovation or cost cutting may fail to reach shareholders.

## The claim that fails:

**“One trading rule that earns strict profit on every possible price history.” (false)**

Proof by contradiction akin Cantor diagonalization, Gödel incompleteness, recursive unsolvability of Turing’s halting problem: For each total deterministic computable rule, construct a *countermarket* on which it cannot win—even though prices move at every date.

# The trader chooses from the history so far

## A trading rule

The rule observes past and current prices and chooses a position before the next price arrives. It cannot use a future price that it has not observed.

## Three choices

**Long:** own shares and gain when their price rises.

**Short:** borrow and sell shares, intending to buy them back later; gain when their price falls.

**Wait:** hold neither a long nor a short position; earn zero trading gain.

## The mathematical setting

The rule is a total deterministic computable map from finite positive rational price histories to rational share positions. It finishes on every input, but no fixed computation deadline is assumed.

## Countermarket construction

# Metamathematical technique: Cantor's diagonal idea

Diagonal reasoning also appears in the work of Gödel and Turing. Suppose there exists a *complete* list containing *all* infinite sequences of zeros and ones. Its beginning might look like this:

|       | bit 1 | bit 2 | bit 3 | bit 4 | ... |
|-------|-------|-------|-------|-------|-----|
| row 1 | 0     | 1     | 1     | 0     | ... |
| row 2 | 1     | 1     | 0     | 1     | ... |
| row 3 | 0     | 1     | 0     | 0     | ... |
| row 4 | 1     | 0     | 1     | 1     | ... |
| ⋮     | ⋮     | ⋮     | ⋮     | ⋮     | ⋮   |

## Change the diagonal entries

Read the highlighted bits and change each 0 to 1, each 1 to 0. The new sequence begins 1, 0, 1, 0, ...

## Why the list cannot be complete?

Proof by contradiction: Suppose it is complete. But the new sequence differs from row 1 at bit 1, from row 2 at bit 2, and from every row at its diagonal position. It is absent from the list. So, the completeness assumption yields a complete contradiction (relative to *tertium non datur*).

Sources: Metamathematics; eg, Noson S. Yanofsky, [A Universal Approach to Self-Referential Paradoxes, Incompleteness and Fixed Points](#) (2003).

# Discounted price: value compared with the bank

## Definition

A *discounted price* is the asset's price divided by the current value of one unit of the comparison bank account. It tells us how many such bank units the asset is worth. The model treats this account as risk-free. All money amounts below are in a fixed currency; eg, Vietnamese dong (VND).

## Example: one bank unit is a deposit of 100 VND

Suppose the account earns 10% over the year, whereas the share sells at 121 VND.

|                | Bank unit | One share | Discounted price |
|----------------|-----------|-----------|------------------|
| At the start   | 100       | 100       | $100/100 = 1$    |
| After one year | 110       | 121       | $121/110 = 1.10$ |

## What the number means

Selling the share now buys 1.10 bank units instead of 1. The share gained 10% *relative to the bank account*. Had the share risen only to 110, its discounted price would still be 1. (This adjusts for bank interest, not inflation.)

# From Cantor's idea to a countermarket

## Cantor's construction

Start with a purportedly complete enumeration.

Use that list itself to construct something that it misses.

Changing the selected bit defeats the completeness claim.

## The trading construction

Start with a proposed universally profitable rule.

Use that rule itself to construct a price path on which it fails.

Opposing each position defeats the universal-profit claim.

## The common idea is diagonal opposition

The counterexample depends on the proposed universal object. Here we construct a market for one fixed trader; we do not need to list all trading programs.

# A Computable Countermarket for Every Trader

## The result, including a moving market

Fix a positive rational starting price and a rational  $0 < \epsilon < 1$ . For every total deterministic computable trader with rational positions, there is a fixed computable path with

$$P_{t+1}/P_t \in \{1 - \epsilon, 1 + \epsilon\}$$

on which accumulated discounted gain is nonpositive at every finite date.

## Every active position loses

Any nonzero position makes accumulated gain strictly negative. A trader that always waits earns zero, although prices keep moving.

## Model and scope

One discounted asset; self-financing trading through shares and a bank account; no fees, borrowing limits, or solvency constraints. This is a worst-case construction, not a model of market equilibrium.

Sources: KS, base manuscript, revised 19 September 2026, subsection “A Computable Countermarket for Every Trader”.

# The countermarket: a constructive proof

## Use the fixed trader's program

Compute its position  $w_t$  from the rational history so far, then set a countermarket price

$$P_{t+1} = \begin{cases} P_t(1 - \epsilon), & w_t > 0, \\ P_t(1 + \epsilon), & w_t < 0, \\ P_t(1 + \epsilon(-1)^t), \text{ or "whatever"} & w_t = 0. \end{cases}$$

At a neutral date (third case), the price rises if  $t$  is even and falls if  $t$  is odd (arbitrary since  $w_t = 0$ ).

## The market always moves

Every return has magnitude  $\epsilon$ . Prices stay positive and rational. Every nonzero position loses; zero holdings earn zero.

## Why the construction works

Adding losses and zeros never gives a positive accumulated gain. Totality makes each next price computable. For the fixed trader, any finite part of this fixed path can be generated before trading begins.

## Worked example: we construct the adverse prices

### The trader's rule is fixed first

The rule is: buy for one period, sell short for the next, then wait. All prices are discounted prices; gains and losses are measured in bank-account units.

### Our construction, step by step

1. The trader buys one share for 100.  
We choose 90 as the next price. Selling the share then loses 10.
2. The trader borrows a share and sells it for 90.  
We choose 99 as the next price. Buying it back to return it loses 9.
3. The trader holds no position.  
At date  $t = 2$ , we raise the price by 10% to 108.9. Its trading gain is still zero.

### Why this is a counterexample

The prices are 100, 90, 99, 108.9; the gains are  $-10, -9, 0$ . Total gain is  $-19$ , although the market moves at every date. This refutes the rule's claim of strict profit on every permitted history.

# Why the countermarket is a fixed computable path

## Put the trader's code inside a price-generating program

1. Start with a positive price and the history built so far.
2. Run the trader's program on that history to obtain its position.
3. Choose a small price fall for a long position, a small rise for a short position, and an alternating-by-date rise or fall for zero holdings.
4. Append that price to the history and repeat.

## Each requested finite part can be calculated

The trader returns its position in finite time. Rational positions, expressed as ratios of integers, can be compared with zero exactly. Each next price is therefore computable.

## The market does not have to observe live orders

For a fixed trader, the generator is fixed. Any finite portion can be computed before trading begins, although computation may be slow.

# Every trader has its own countermarket

## The order of the claim matters

Proposed guarantee: “There is one trader that profits on every path.”

Countermarket result: “For each proposed trader, construct a path on which every nonzero position loses.”

## Another trader may win on that same record

For one price rise, a trader holding one share gains and a trader shorting one share loses. A price fall reverses their fortunes. The construction does not claim to make every other trader lose on that path.

## The stronger contribution

A constant discounted price already gives every trader zero trading gain. Our construction also works when flat paths are excluded: every return has fixed nonzero magnitude, and every active position of the selected trader loses. The path need not be typical, an equilibrium, or arbitrage-free.

# Two sources of randomness

## Trader side: a private coin chooses positions

After observing prices, the trader might toss a private coin: heads means hold one share; tails means short one share. The program and price history alone no longer determine its next position. Our deterministic countermarket cannot calculate and oppose that hidden draw.

## Market side: randomness determines price movements

The next price may instead be generated randomly, independently of the trader's private seed. Both sources can occur together. A passive market does not react to the trader's orders.

## Randomness alone does not eliminate profits

Randomizing positions need not destroy an existing edge. Random market movements can still have positive expected returns. Zero expected gain requires additional fairness assumptions.

Sources: KS, subsection "Worst-Case Value and Private Randomization".

# An independent fair market gives zero expected gain

## The market's coin chooses the price movement

The trader first chooses its position, possibly at random. An independent fair coin then moves the discounted price from 100 to 90 or 110, each with probability one half. The market's coin is independent of the past and of the trader's private coin.

## Average over the market's two equally likely moves

| Position         | Price 90 | Price 110 | Expected gain |
|------------------|----------|-----------|---------------|
| Own one share    | -10      | +10       | 0             |
| Short one share  | +10      | -10       | 0             |
| Hold no position | 0        | 0         | 0             |

## The result applies to deterministic and randomized traders

Every position has expected gain zero, however selected. Repeat independent fair 10% rises or falls from each current price. Over any fixed finite horizon, expected total gain is zero, provided gains and losses have finite averages.

Individual runs may win or lose. Zero expected gain is not a claim that realized profits converge to zero in the long run.

Sources: KS, proposition "Private Randomization Does Not Create a Guarantee".

Some related ideas

# Gold: can history reveal the market's rule?

## Learning in the limit

After each new observation, a learner proposes a program explaining the whole sequence. Success means eventually settling on one correct program and never changing it again. The learner need not know when it has succeeded.

## Gold's negative result

No algorithm achieves this for every computable sequence of zeros and ones. For each learner, some computable history defeats this requirement. Failure means no eventual correct identification, rather than a specified number of losing trades.

## Restrictions can make learning possible

For a uniformly effective indexed family of total computable rules, identification in the limit is possible: keep the first rule consistent with all observations so far. What matters is which market rules the learner admits.

Sources: E. M. Gold, *Limiting recursion* (1965); *Language identification in the limit* (1967). Related prediction construction: Legg, *Is there an Elegant Universal Theory of Prediction?* (2006).

# Turing: will the price ever rise?

## An artificial market controlled by a program

Run any program with fixed input, one instruction per day. Start the price at 1.

- ▶ While the program keeps running, the price stays at 1.
- ▶ If the program stops, the price becomes 2 and stays there.

## The price on a specified day is computable

| Program's behaviour    | Day 1 | Day 2 | Day 3 | Day 4 | ... |
|------------------------|-------|-------|-------|-------|-----|
| Stops at instruction 3 | 1     | 1     | 2     | 2     | ... |
| Never stops            | 1     | 1     | 1     | 1     | ... |

For day 100, simulate 100 instructions: report 2 if the program has stopped, otherwise 1. This calculation always finishes.

## But will the price ever rise?

The price rises exactly when the program stops. Deciding this for every program and input would solve the *Halting Problem*. No algorithm always finishes with the correct yes-or-no answer.

Sources: A. M. Turing, [On computable numbers \(1936–37\)](#). Market construction: the paper.

# Radó's Busy Beaver: how long must we wait?

## A finite maximum with no computable general bound

Fix a *universal* programming language, capable of simulating any ordinary program. Include any required input in its program description. Among programs of at most  $n$  bits that eventually stop, take the longest running time. This is the runtime *Busy Beaver* value, written  $BB(n)$ . Here  $n$  measures program length, not elapsed time.

## Return to the price 1 that may become 2

A short program can run for an extraordinarily long time before stopping. Its market can therefore remain flat through a long observation period and change only later.

## Why no waiting rule suffices

No computable deadline based only on program length covers every eventual halt. Such a deadline would solve Turing's problem: wait until it expires. Some finite cases can be settled, but they give no general extrapolation rule.

Sources: T. Radó, *On non-computable functions* (1962); G. J. Chaitin, *Computing the busy beaver function* (1987).

# From random trials to ML randomness

## Two levels

*Process randomness*: how individual bits are generated. *Martin-Löf (ML) randomness*: a property of an infinite record relative to a specified probability law.

## The sufficient condition

Independent fair bits produce a fair-coin ML-random sequence **with probability one**.

Independence alone is insufficient if the bits are biased. Fair one-bit probabilities alone also fail: repeating one fair bit forever gives a constant record.

## The practical qualification

More generally, any computable process law produces records ML-random for that law almost surely; dependence is allowed. Finite tests can challenge a source model, but cannot certify the randomness of its infinite output. Probability one is not a guarantee for every outcome.

Sources: P. Martin-Löf, *The definition of random sequences* (1966), Sec. IV.

# Borel normality is weaker than randomness

## Correct frequencies at every block length

A base- $b$  record is *normal* if every word of length  $k$  has limiting frequency  $b^{-k}$ , for every  $k \geq 1$ . ML randomness for uniform digits implies normality.

## Champernowne's decimal

0.123456789101112131415...

Concatenate the positive integers. This decimal is normal in base ten, yet computable: its digits are prescribed by a simple rule. It is therefore not ML-random for the uniform decimal law.

## Why verification is difficult

Normality concerns all block lengths and infinite limits. No finite frequency check proves it: every finite prefix admits both normal and nonnormal continuations. A mathematical construction can establish what sampling alone cannot.

Sources: D. G. Champernowne, *The Construction of Decimals Normal in the Scale of Ten* (1933).

# Ramsey: order appears even without a predictive law

## An unavoidable pattern

Join every pair among six objects and color each link red or blue. Whatever the coloring, three objects have all three links the same color. This is a basic finite case of Ramsey's theorem.

## Repeated symbols at equally spaced dates

Van der Waerden's theorem gives a related result: in a sufficiently long record with finitely many symbols, some symbol repeats at equally spaced positions for any prescribed finite number of repetitions.

## The finite-pattern trap

The selected dates need not be consecutive, the returns need not be positive, and the pattern need not continue. Calude and Longo emphasize that large data sets force some regularities even when the records resist compression. Such patterns coexist with Martin-Löf randomness.

Sources: Graham, Rothschild & Spencer, *Ramsey Theory* (1990); Graham & Spencer (1990); Calude & Longo, *The deluge of spurious correlations in big data* (2017).

# Time reversal as a search for counterexamples

## Hold the trading rule fixed, change the record

Test it on the illustrative prices 100, 110, 120 and then rerun it from scratch on 120, 110, 100. At every date it may use only the history available on that run. Reversing the original orders would give it information from the future.

## If the initial price must stay fixed

Use  $P^{\text{rev}}(t) = P(0)P(T - t)/P(T)$ , provided it stays in the class. This rescales the reversed record; a price-level-sensitive rule may act differently.

## One failure refutes an all-path claim

If both records belong to the claimed market class, a nonpositive gain on either refutes strict profit on every path. Reversal alone does not prove that a rule must fail on one of the pair. It supplies a stress test, whereas diagonalization supplies the counterexample.

Sources: KS, "Practical Robustness Diagnostics: Time Reversal and Permutations", Reversal Falsification Test.

# Beyond time reversal: reshuffling the record

## Algorithmic and random permutations

A *permutation* rearranges the same finite record of discounted price levels. Use a fixed algorithm or a random shuffle. Rerun the trader from scratch, giving it only the past available in the new order.

## A necessary condition for a universal profit guarantee

A strategy claiming strict profit on every permitted path **must remain profitable under every reshuffling that stays within its claimed market class**. The amount of profit may change. One zero or negative gain refutes that universal claim.

## What this requirement means

A strategy exploiting a particular trend may fail when reshuffling destroys that trend. Such failure limits its scope. Conversely, passing a finite collection of shuffles cannot establish universal profitability.

*Literary analogy:* Greg Egan's *Permutation City* (1994) explores the ordering of computational states.

Sources: Greg Egan, *Permutation City* (1994); Egan's *Dust Theory* FAQ, questions 1–2.

# Summary of Countermarkets

## A constructive worst-case result

For every total deterministic computable rule on rational histories, there is a positive computable path with a fixed nonzero return magnitude on which its self-financing gain is nonpositive at every finite horizon.

## What the result does and does not establish

Every active position loses; waiting earns zero while the market moves. The path depends on the chosen trader and need not be arbitrage-free. This is an expository construction, not a new general impossibility theorem.

## Conditional success remains possible

Private randomization and random markets do not generally eliminate profits. Information, luck, or favorable market conditions can produce gains. Specify the market law, horizon, costs, and benchmark; these gains are compatible with the failure of an all-path guarantee.

Thank you for your attention